

DOCUMENTACIÓN TÉCNICA

Manejo de Visión del Unitree R1

Visor de cámara frontal mediante Unitree SDK2 Python, DDS y OpenCV

Campo	Detalle
Paquete / Script	r1_camera_sdk.py y start_camera.sh
Versión	1.0
Plataforma	Ubuntu 20.04, Python 3, ROS2 Foxy como entorno de red y dependencias
Robot / Hardware	Unitree R1 con cámara frontal y conexión de red al computador de ejecución
SDK / Framework	unitree_sdk2_python, DDS/CycloneDDS, OpenCV, NumPy
Script base	Implementación RPC con VideoClient.GetImageSample()
Autor	Robotics 4.0
Fecha	2026-06-26

Robotics 4.0 – Equipo de desarrollo

Documento reproducible para instalación, operación, depuración y extensión del módulo de visión.

0. Índice

1. Descripción General	4
1.1. Problema que reemplaza	4
1.2. Componentes desarrollados	5
1.3. Entradas y salidas	5
1.4. Usuarios previstos	6
2. Arquitectura del Sistema	6
2.1. Descripción por capas	6
2.2. Diagrama textual del flujo	6
2.3. Canales de comunicación	7
2.4. Diferencia frente al enfoque ROS2 puro	7
3. Requisitos y Dependencias	7
3.1. Requisitos principales	8
3.2. Instalación mínima de dependencias	8
3.3. Verificación de SDK2 Python	8
3.4. Configuración de CycloneDDS	9
4. Organización de Archivos	9
4.1. Estructura recomendada	9
4.2. Archivos obligatorios para operación	10
4.3. Archivos de referencia ROS2	10
4.4. Reglas de ubicación	10
5. Formato de Entrada	11
5.1. Argumentos de terminal	11
5.2. Ejemplos de entrada	11
5.3. Controles de teclado	11
5.4. Variable de entorno	11
6. Código Fuente y Funcionamiento Interno	12

6.1. Resumen de módulos	12
6.2. Importación del SDK y configuración DDS	12
6.3. Resoluciones y modos	13
6.4. Redimensionamiento y rotulado	13
6.5. Modo multi-vista	13
6.6. Inicialización del cliente de video	14
6.7. Lectura, decodificación y visualización	14
6.8. Cambio de modo en tiempo real	14
6.9. Wrapper de ejecución	15
6.10. Ruta ROS2 no usada	15
7. Flujo de Ejecución	15
7.1. Preparación del entorno	15
7.2. Validación de archivos	16
7.3. Permisos del wrapper	16
7.4. Ejecución estándar	16
7.5. Ejecución con modo e interfaz explícitos	16
7.6. Ejecución directa sin wrapper	16
7.7. Verificación del resultado	16
7.8. Cierre seguro	16
8. Seguridad Operacional	17
8.1. Condiciones antes de ejecutar	17
8.2. Riesgos principales	17
8.3. Advertencia crítica	17
9. Guía de Uso Rápido	18
10. Problemas Conocidos y Soluciones	18
11. Extensión y Mantenimiento	19
11.1. Agregar nuevas resoluciones	19
11.2. Guardar frames en disco	20
11.3. Agregar procesamiento de visión	20
11.4. Publicar frames a ROS2	20
11.5. Partes que no deben modificarse sin precaución	20

12. Resumen de Actividades

21

13. Anexo: Sustitución de Diapositivas de Visión

21

13.1. Texto sugerido para diapositiva introductoria	21
13.2. Texto sugerido para diapositiva de ejecución	21
13.3. Texto sugerido para controles	22
13.4. Código breve para imagen de diapositiva	22



1. Descripción General

Esta documentación describe la implementación de visión usada para visualizar la cámara frontal del robot Unitree R1. El enfoque final no utiliza un pipeline directo de GStreamer con multicast UDP. En su lugar, emplea una consulta RPC mediante `VideoClient.GetImageSample()` desde `unitree_sdk2_python`; cada muestra recibida llega como imagen JPEG, se decodifica con OpenCV y se muestra en una ventana local.

La herramienta resuelve el problema de acceso práctico al video del R1 cuando el tópicos ROS2 `/frontvideostream` no resulta usable desde Python por incompatibilidad de representación de datos en CycloneDDS. Por esta razón, el flujo recomendado para operación es el script `r1_camera_sdk.py`, ejecutado de forma directa o por medio del wrapper `start_camera.sh`.

Idea central del sistema: el robot entrega frames JPEG a través del servicio DDS `videohub`; el computador cliente los solicita con el SDK2 Python, los convierte a arreglos NumPy, los decodifica con `cv2.imdecode()` y los despliega en tiempo real.

1.1 Problema que reemplaza

El flujo previo de visión se basaba en GStreamer y asumía parámetros específicos del G1, como multicast UDP, puerto fijo, codificación H.264 y resolución predeterminada. Para el R1, esa información no debe trasladarse directamente. El desarrollo actual reemplaza ese enfoque por una solución basada en el cliente de video del SDK2 Python, con selección de resolución en cliente y modo multi-vista.

1.2 Componentes desarrollados

Componente	Tipo	Descripción	Estado
<code>r1_camera_sdk.py</code>	Script principal	Solicita frames JPEG mediante <code>VideoClient</code> , decodifica las imágenes y muestra una ventana OpenCV con selección de resolución.	Vigente
<code>start_camera.sh</code>	Wrapper Bash	Define el modo inicial, exporta <code>CYCLONEDDS_URI</code> y ejecuta el script principal.	Vigente
<code>README.md</code>	Guía breve	Resume requisitos, uso, controles, arquitectura y motivo por el cual se evita <code>/frontvideostream</code> .	Vigente
<code>src/r1_camera_viewer/</code>	Paquete ROS2	Paquete <code>ament_python</code> que intenta consumir <code>/frontvideostream</code> ; se conserva como referencia, pero no es el camino operativo.	Referencia
<code>camera_viewer_node.py</code>	Nodo ROS2	Nodo que se suscribe a <code>/frontvideostream</code> con <code>Go2FrontVideoData</code> . No es la ruta recomendada por la incompatibilidad detectada.	No usado
<code>camera.launch.py</code>	Launch ROS2	Permite lanzar el nodo ROS2 con parámetro de resolución. Se conserva para trazabilidad técnica.	No usado

1.3 Entradas y salidas

Elemento	Entrada	Salida / efecto
Modo de visualización	720p, 360p, 180p o all	Define el tamaño mostrado o el mosaico multi-vista.
Interfaz de red	Nombre de interfaz, por ejemplo <code>enp0s31f6</code> o <code>eth0</code> ; puede omitirse para autodetección del SDK.	Inicializa el canal DDS hacia el robot.
Teclado	Teclas 1, 2, 3, a, ESC.	Cambia resolución en tiempo real o cierra la ventana.
Robot R1	Frames JPEG obtenidos por <code>GetImageSample()</code> .	Imagen decodificada y visualizada en OpenCV.

1.4 Usuarios previstos

Esta herramienta está orientada a integrantes del equipo de desarrollo que necesiten verificar la cámara frontal del R1, validar conectividad DDS, observar video en vivo o preparar una capa posterior de percepción computacional. No está diseñada para controlar locomoción, brazos ni actuadores del robot.

2. Arquitectura del Sistema

2.1 Descripción por capas

El sistema opera en cuatro capas: hardware del R1, comunicación DDS, script cliente y visualización OpenCV. La cámara frontal del robot genera imágenes que el servicio de video expone al cliente. El computador local inicializa el canal de comunicación con `ChannelFactoryInitialize()` y consulta cada frame con `VideoClient.GetImageSample()`.

Capa / Módulo	Función	Entrada	Salida
Hardware R1	Captura imagen desde el sensor frontal.	Escena física frente al robot.	Frame JPEG disponible en el servicio de video.
DDS / videohub	Expone el servicio de imagen usado por VideoClient.	Solicitud RPC del cliente.	Paquete de bytes JPEG.
r1_camera_sdk.py	Inicializa SDK, solicita frames, decodifica y administra modos de vista.	Interfaz de red, modo inicial, frames JPEG.	Frame BGR o mosaico de resoluciones.
OpenCV GUI	Renderiza la imagen en una ventana local.	Matriz NumPy BGR.	Ventana R1 Camera.

2.2 Diagrama textual del flujo

Listing 1: Flujo lógico de datos del visor de cámara R1.

```

Camara frontal R1
-> servicio DDS videohub
  -> VideoClient.GetImageSample()
    -> bytes JPEG
      -> np.frombuffer()
        -> cv2.imdecode()
          -> resize / build_all_view
            -> cv2.imshow()
  
```

2.3 Canales de comunicación

Canal / Interfaz	Uso	Entorno	Observación
Ethernet o interfaz de red del robot	Transporte físico de comunicación entre PC y R1.	Computador local conectado al robot.	Se puede pasar como argumento del script.
CYCLONEDDS_URI	Ubicación del archivo de configuración CycloneDDS.	Shell del usuario.	El wrapper apunta a \$HOME/unitree_ros2/ros_config.x
ChannelFactoryInitiali iface)	Inicialización de comunicación SDK2 Python.	r1_camera_sdk.py.	Si iface está vacío, el SDK intenta inicializar sin interfaz explícita.
VideoClient.GetImageSa	Solicitud del frame al servicio de video.	SDK2 Python.	Retorna código de estado y datos JPEG.
OpenCV HighGUI	Visualización local.	Computador con entorno gráfico.	Requiere servidor gráfico disponible.

2.4 Diferencia frente al enfoque ROS2 puro

El paquete ROS2 r1_camera_viewer fue construido como alternativa para suscribirse al tópico /frontvideostream. La implementación define un nodo llamado r1_camera_viewer, declara un parámetro resolution y decodifica los campos video720p, video360p o video180p. Sin embargo, el README del proyecto documenta que este tópico no fue usable desde Python por un problema de compatibilidad con CycloneDDS 0.10.2 y XCDRV2. Por esa razón, el enfoque RPC con VideoClient queda como ruta operativa.

3. Requisitos y Dependencias

3.1 Requisitos principales

Requisito	Versión / Valor	Verificación
Sistema operativo	Ubuntu 20.04 recomendado para entorno ROS2 Foxy del equipo.	<code>lsb_release -a</code>
Python	Python 3.8 o compatible con ROS2 Foxy.	<code>python3 -version</code>
ROS2	Foxy.	<code>echo \$ROS_DISTRO</code>
RMW	<code>rmw_cyclonedds_cpp</code> .	<code>echo \$RMW_IMPLEMENTATION</code>
SDK2 Python	Carpeta <code>~/unitree_sdk2_python</code> .	<code>test -d ~/unitree_sdk2_python && echo OK</code>
CycloneDDS config	<code>~/unitree_ros2/ros_config.xml</code>	<code>test -f ~/unitree_ros2/ros_config.xml && echo OK</code>
OpenCV	<code>python3-opencv</code> o <code>cv2</code> instalable.	<code>python3 -c "import cv2; print(cv2.__version__)"</code>
NumPy	<code>python3-numpy</code> .	<code>python3 -c "import numpy; print(numpy.__version__)"</code>
Red con R1	Interfaz activa en la subred del robot.	<code>ip link show</code> y prueba de conectividad.

3.2 Instalación mínima de dependencias

Listing 2: Instalación de paquetes base para el visor.

```
sudo apt update
sudo apt install -y python3-opencv python3-numpy
sudo apt install -y ros-foxy-rmw-cyclonedds-cpp
```

3.3 Verificación de SDK2 Python

El script principal inserta `~/unitree_sdk2_python` al inicio de `sys.path`. Por tanto, esa carpeta debe existir y contener el paquete `unitree_sdk2py`.

Listing 3: Validación de ruta y módulo SDK2 Python.

```
test -d ~/unitree_sdk2_python && echo "SDK2 Python encontrado"
python3 - <<'PY'
import sys, os
sys.path.insert(0, os.path.expanduser("~/unitree_sdk2_python"))
from unitree_sdk2py.core.channel import ChannelFactoryInitialize
from unitree_sdk2py.go2.video.video_client import VideoClient
print("OK: SDK2 Python y VideoClient importados")
PY
```

3.4 Configuración de CycloneDDS

El wrapper `start_camera.sh` exporta la variable:

```
export CYCLONEDDS_URI="file://$HOME/unitree_ros2/ros_config.xml"
```

Antes de ejecutar, verifique que el archivo exista y que la interfaz de red configurada corresponda a la interfaz realmente conectada al R1.

Listing 4: Verificación de configuración DDS.

```
test -f ~/unitree_ros2/ros_config.xml && echo "ros_config.xml encontrado"
grep -n "NetworkInterface\|name" ~/unitree_ros2/ros_config.xml || true
ip -br link
ip -br addr
```

4. Organización de Archivos

4.1 Estructura recomendada

La estructura de trabajo recomendada es la siguiente:

Listing 5: Árbol de directorios del workspace de cámara R1.

```
camara_r1_ws/
|-- r1_camera_sdk.py           # script principal operativo
|-- start_camera.sh           # wrapper de ejecución
|-- README.md                 # guía corta del workspace
'-- src/
    '-- r1_camera_viewer/      # paquete ROS2 de referencia, no usado en operación
        |-- package.xml
        |-- setup.py
        |-- setup.cfg
        |-- launch/
        |   '-- camera.launch.py
    '-- r1_camera_viewer/
        '-- camera_viewer_node.py
```

4.2 Archivos obligatorios para operación

Archivo	Función	Obligatorio
<code>r1_camera_sdk.py</code>	Contiene la lógica de conexión, lectura de frames, decodificación, cambio de resolución y visualización.	Sí
<code>start_camera.sh</code>	Simplifica la ejecución, exporta <code>CYCLONEDDS_URI</code> y pasa argumentos al script.	Recomendado
<code>~/unitree_ros2/ros_conf</code>	Configuración de CycloneDDS usada por el SDK.	Sí
<code>~/unitree_sdk2_python</code>	Repositorio SDK2 Python importado por el script.	Sí

4.3 Archivos de referencia ROS2

Archivo	Uso	Estado
<code>camera_viewer_node.py</code>	Nodo ROS2 que intenta suscribirse a <code>/frontvideostream</code> y decodificar campos de resolución.	Referencia técnica
<code>camera.launch.py</code>	Launch con argumento <code>resolution</code> .	Referencia técnica
<code>package.xml</code>	Declara dependencias ROS2 como <code>rclpy</code> , <code>unitree_go</code> , <code>python3-opencv</code> y <code>python3-numpy</code> .	Referencia técnica
<code>setup.py</code>	Define el entry point <code>camera_viewer</code> .	Referencia técnica

4.4 Reglas de ubicación

- `r1_camera_sdk.py` y `start_camera.sh` deben estar en la misma carpeta, porque el wrapper ejecuta el script con `$(dirname "$0")/r1_camera_sdk.py`.
- La ruta `~/unitree_sdk2_python` está codificada en el script principal. Si el SDK está en otra ubicación, debe modificarse la variable `_SDK_PATH`.
- La ruta `~/unitree_ros2/ros_config.xml` se usa tanto en el script como en el wrapper. Si la configuración DDS está en otra ruta, debe actualizarse `CYCLONEDDS_URI`.
- Se recomienda evitar espacios en nombres de carpeta para reducir problemas en ejecución por terminal.

5. Formato de Entrada

La herramienta no consume archivos JSON, YAML ni CSV. Sus entradas son argumentos de terminal, variables de entorno y comandos de teclado durante la ejecución.

5.1 Argumentos de terminal

Argumento	Valores válidos	Default	Descripción
modo	720p, 360p, 180p, all	720p	Resolución inicial o mosaico multi-vista.
iface	Nombre de interfaz, por ejemplo enp0s31f6 o eth0.	Vacío	Interfaz de red usada para DDS. Si se omite, el SDK inicializa sin interfaz explícita.

5.2 Ejemplos de entrada

```
./start_camera.sh
./start_camera.sh all
./start_camera.sh 360p enp0s31f6
python3 r1_camera_sdk.py enp0s31f6 720p
python3 r1_camera_sdk.py "" all
```

5.3 Controles de teclado

Tecla	Acción
1	Cambia a vista 720p 1280 × 720.
2	Cambia a vista 360p 640 × 360.
3	Cambia a vista 180p 320 × 180.
a / A	Cambia a modo all, con mosaico de las tres resoluciones.
ESC	Cierra el visor y destruye las ventanas OpenCV.

5.4 Variable de entorno

Variable	Valor esperado	Uso
CYCLONEDDS_URI	file://\$HOME/unitree_ros2/ros_c	Indica al middleware dónde encontrar la configuración DDS para comunicarse con el robot.

6. Código Fuente y Funcionamiento Interno

6.1 Resumen de módulos

Elemento	Responsabilidad	Entradas	Salidas / Efecto
_SDK_PATH	Agrega el SDK2 Python al sys.path.	Ruta local ~/unitree_sdk2_pyth	Permite importar unitree_sdk2py.
SIZES	Define resoluciones de visualización.	Nombre de modo.	Ancho y alto esperados.
resize()	Redimensiona un frame si no coincide con el modo seleccionado.	Frame NumPy y resolución.	Frame redimensionado.
draw_label()	Añade texto de resolución sobre la imagen.	Frame y etiqueta.	Frame rotulado.
build_all_view()	Construye mosaico de 720p, 360p y 180p.	Frame nativo 720p.	Imagen compuesta.
main()	Inicializa canal, cliente de video y bucle principal.	iface, mode.	Ventana OpenCV con stream en vivo.

6.2 Importación del SDK y configuración DDS

El script no depende de una instalación global del paquete; inserta manualmente ~/unitree_sdk2_python en sys.path. También define un valor por defecto para CYCLONEDDS_URI si la variable no fue exportada antes.

Listing 6: Carga de SDK2 Python y configuración DDS.

```

_SDK_PATH = os.path.expanduser("~/unitree_sdk2_python")
if _SDK_PATH not in sys.path:
    sys.path.insert(0, _SDK_PATH)

os.environ.setdefault(
    "CYCLONEDDS_URI",
    f"file://{os.path.expanduser('~')}/unitree_ros2/ros_config.xml"
)

from unitree_sdk2py.core.channel import ChannelFactoryInitialize
from unitree_sdk2py.go2.video.video_client import VideoClient

```

6.3 Resoluciones y modos

El cliente obtiene el frame nativo en 720p. Las resoluciones inferiores se generan localmente mediante `cv2.resize()`.

Listing 7: Modos de resolución disponibles.

```
SIZES = {
    "720p": (1280, 720),
    "360p": (640, 360),
    "180p": (320, 180),
}

MODES = ["720p", "360p", "180p", "all"]
MODE_KEYS = {ord("1"): "720p", ord("2"): "360p", ord("3"): "180p", ord("a"): "all"}
```

6.4 Redimensionamiento y rotulado

Listing 8: Funciones de ajuste visual del frame.

```
def resize(frame: np.ndarray, res: str) -> np.ndarray:
    w, h = SIZES[res]
    if frame.shape[1] == w and frame.shape[0] == h:
        return frame
    return cv2.resize(frame, (w, h), interpolation=cv2.INTER_AREA)

def draw_label(frame: np.ndarray, text: str) -> np.ndarray:
    out = frame.copy()
    cv2.putText(out, text, (8, 24), cv2.FONT_HERSHEY_SIMPLEX,
                0.7, (0, 0, 0), 3, cv2.LINE_AA)
    cv2.putText(out, text, (8, 24), cv2.FONT_HERSHEY_SIMPLEX,
                0.7, (255, 255, 255), 1, cv2.LINE_AA)
    return out
```

6.5 Modo multi-vista

El modo `all` toma un frame base de 720p, genera las versiones 360p y 180p, y las compone en un mosaico. Este enfoque evita pedir tres streams al robot; todo se deriva del mismo frame nativo.

Listing 9: Construcción del mosaico de resoluciones.

```
def build_all_view(frame720: np.ndarray) -> np.ndarray:
    f720 = draw_label(frame720, "720p 1280x720")
    f360 = draw_label(resize(frame720, "360p"), "360p 640x360")
    f180 = draw_label(resize(frame720, "180p"), "180p 320x180")

    row_h = f360.shape[0]
    row_w = f720.shape[1]
    block_w = row_w - f360.shape[1]
    block = np.zeros((row_h, block_w, 3), dtype=np.uint8)

    y_off = (row_h - f180.shape[0]) // 2
    x_off = (block_w - f180.shape[1]) // 2
    block[y_off:y_off + f180.shape[0], x_off:x_off + f180.shape[1]] = f180

    bottom = np.hstack([f360, block])
```

```
return np.vstack([f720, bottom])
```

6.6 Inicialización del cliente de video

La función `main()` interpreta los argumentos, inicializa el canal DDS y crea el cliente de video.

Listing 10: Inicialización de canal y cliente de video.

```
iface = sys.argv[1] if len(sys.argv) > 1 else ""
mode = sys.argv[2] if len(sys.argv) > 2 else "720p"
if mode not in MODES:
    mode = "720p"

if iface:
    ChannelFactoryInitialize(0, iface)
else:
    ChannelFactoryInitialize(0)

client = VideoClient()
client.SetTimeout(3.0)
client.Init()
```

6.7 Lectura, decodificación y visualización

El bucle principal llama a `GetImageSample()`, verifica el código de retorno, convierte el buffer a `uint8`, decodifica el JPEG y actualiza la ventana.

Listing 11: Bucle operativo principal.

```
while True:
    code, data = client.GetImageSample()
    if code != 0:
        print(f"Error frame: code={code}", flush=True)
        if cv2.waitKey(100) == 27:
            break
        continue

    raw = np.frombuffer(bytes(data), dtype=np.uint8)
    frame = cv2.imdecode(raw, cv2.IMREAD_COLOR)
    if frame is None:
        continue

    if mode == "all":
        display = build_all_view(frame)
    else:
        display = draw_label(resize(frame, mode), f"{mode} {SIZES[mode][0]}x{SIZES[mode][1]}")

    cv2.imshow("R1 Camera", display)
```

6.8 Cambio de modo en tiempo real

Listing 12: Control de teclado del visor.

```
key = cv2.waitKey(20) & 0xFF
if key == 27:
```

```
break
if key in MODE_KEYS:
    mode = MODE_KEYS[key]
    print(f"Modo: {mode}", flush=True)
```

6.9 Wrapper de ejecución

start_camera.sh centraliza los valores por defecto y reduce errores al ejecutar.

Listing 13: Wrapper Bash de ejecución.

```
MODE="${1:-720p}"
IFACE="${2:-}"

export CYCLONEDDS_URI="file://$HOME/unitree_ros2/ros_config.xml"

python3 "$(dirname "$0")/r1_camera_sdk.py" "$IFACE" "$MODE"
```

6.10 Ruta ROS2 no usada

El nodo ROS2 conserva valor documental. Su lógica base es suscribirse a /frontvideostream, seleccionar el campo de resolución y decodificar el frame. No debe tratarse como el flujo principal mientras persista la incompatibilidad indicada.

Listing 14: Lógica resumida del nodo ROS2 de referencia.

```
self.subscription = self.create_subscription(
    Go2FrontVideoData,
    '/frontvideostream',
    self._callback,
    10
)

if self._res == '720p':
    raw = msg.video720p
elif self._res == '180p':
    raw = msg.video180p
else:
    raw = msg.video360p
```

7. Flujo de Ejecución

7.1 Preparación del entorno

Listing 15: Preparación del entorno antes de ejecutar.

```
source /opt/ros/foxy/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
export CYCLONEDDS_URI="file://$HOME/unitree_ros2/ros_config.xml"
cd ~/camara_r1_ws
```


7.2 Validación de archivos

Listing 16: Validación rápida de archivos obligatorios.

```
ls -l r1_camera_sdk.py start_camera.sh
python3 -m py_compile r1_camera_sdk.py
test -f ~/unitree_ros2/ros_config.xml && echo "DDS config OK"
test -d ~/unitree_sdk2/python && echo "SDK path OK"
```

7.3 Permisos del wrapper

```
chmod +x start_camera.sh
```

7.4 Ejecución estándar

Listing 17: Ejecución estándar con wrapper.

```
./start_camera.sh
```

7.5 Ejecución con modo e interfaz explícitos

Listing 18: Ejecución indicando resolución e interfaz.

```
./start_camera.sh 720p enp0s31f6
./start_camera.sh 360p enp0s31f6
./start_camera.sh all enp0s31f6
```

7.6 Ejecución directa sin wrapper

Listing 19: Ejecución directa del script principal.

```
python3 r1_camera_sdk.py enp0s31f6 720p
python3 r1_camera_sdk.py enp0s31f6 all
```

7.7 Verificación del resultado

La ejecución correcta debe producir una ventana llamada R1 Camera. En terminal debe aparecer el texto de ayuda con los controles. Si no se especifica modo, el sistema inicia en 720p. Si el modo pasado no está en la lista de modos válidos, el script vuelve automáticamente a 720p.

7.8 Cierre seguro

El cierre normal se realiza con ESC dentro de la ventana OpenCV. Al salir, el script ejecuta `cv2.destroyAllWindows()`.

No usar Ctrl+C como cierre normal. Puede usarse como último recurso si la ventana queda bloqueada, pero el cierre esperado es con ESC para permitir que OpenCV destruya correctamente la ventana.

8. Seguridad Operacional

Aunque esta herramienta no envía comandos de movimiento, sí se ejecuta conectada al robot físico. Su uso debe tratarse como una operación sobre hardware real.

8.1 Condiciones antes de ejecutar

- Verificar que el R1 esté encendido y conectado a la misma red que el computador.
- Confirmar que la interfaz de red configurada en `ros_config.xml` corresponde a la interfaz física usada.
- No iniciar pruebas de locomoción, brazos o bajo nivel simultáneamente si el objetivo es depurar cámara.
- Asegurar que la cámara tenga campo visual libre y que el robot esté estable.
- Usar el visor como herramienta de monitoreo, no como única referencia para navegación o seguridad.

8.2 Riesgos principales

Riesgo	Causa	Mitigación
Latencia visual	Red saturada, modo a11 o bajo rendimiento gráfico.	Usar 720p o 360p; evitar tareas pesadas simultáneas.
Sin frames	Interfaz incorrecta, CYCLONEDDS_URI inválido o robot fuera de red.	Verificar IP, interfaz y archivo <code>ros_config.xml</code> .
Ventana congelada	Error de GUI, falta de entorno gráfico o frame inválido.	Cerrar con ESC; si no responde, terminar proceso desde terminal.
Confusión con flujo ROS2	Intentar usar <code>/frontvideostream</code> como ruta principal.	Usar <code>r1_camera_sdk.py</code> mientras el tópico siga siendo incompatible.

8.3 Advertencia crítica

La herramienta de visión no sustituye una parada de emergencia. Si se está usando la cámara durante pruebas físicas, debe existir supervisión directa del robot y acceso inmediato al control físico o parada de emergencia. No se debe depender exclusivamente del video para tomar decisiones de seguridad.

9. Guía de Uso Rápido

1. Conectar el computador a la red del R1.
2. Verificar interfaz con `ip -br addr`.
3. Verificar que exista `~/unitree_ros2/ros_config.xml`.
4. Abrir el workspace: `cd ~/camara_r1_ws`.
5. Dar permisos al wrapper: `chmod +x start_camera.sh`.
6. Ejecutar: `./start_camera.sh 720p enp0s31f6`.
7. Cambiar resolución con 1, 2, 3 o a.
8. Cerrar con ESC.

Listing 20: Secuencia rápida de ejecución.

```
source /opt/ros/foxy/setup.bash
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
cd ~/camara_r1_ws
chmod +x start_camera.sh
./start_camera.sh all enp0s31f6
```

10. Problemas Conocidos y Soluciones

Problema	Causa probable	Solución
ModuleNotFoundError: unitree_sdk2py	La carpeta <code>~/unitree_sdk2_python</code> no existe, está incompleta o el paquete no está en <code>sys.path</code> .	Clonar o ubicar el SDK en <code>~/unitree_sdk2_python</code> . Verificar importación con Python.
ModuleNotFoundError: cv2	OpenCV no está instalado.	Ejecutar <code>sudo apt install python3-opencv</code> .
ModuleNotFoundError: numpy	NumPy no está instalado.	Ejecutar <code>sudo apt install python3-numpy</code> .

Problema	Causa probable	Solución
No aparece ventana	No hay entorno gráfico o se ejecuta por SSH sin X forwarding.	Ejecutar localmente en un escritorio con GUI o configurar X forwarding correctamente.
Error frame: code=... repetido	El cliente no recibe frames válidos del servicio de video.	Revisar red, interfaz, robot encendido y CYCLONEDDS_URI. Probar con interfaz explícita.
Imagen congelada o lenta	PC con carga alta, modo all, red inestable o ventanas OpenCV saturadas.	Probar 360p; cerrar otros procesos; usar interfaz cableada.
El modo ingresado no funciona	Se pasó un valor fuera de 720p, 360p, 180p, all.	El script vuelve a 720p. Ejecutar con un modo válido.
Permission denied: ./start_camera.sh	El wrapper no tiene permiso de ejecución.	Ejecutar <code>chmod +x start_camera.sh</code> .
ros_config.xml no encontrado	La configuración DDS no está en <code>~/unitree_ros2/ros_config.xml</code> .	Copiar el archivo a esa ruta o cambiar CYCLONEDDS_URI.
El tópico /frontvideostream no funciona desde Python	Incompatibilidad de representación de datos con CycloneDDS 0.10.2 según la evidencia del proyecto.	Usar <code>r1_camera_sdk.py</code> con <code>VideoClient.GetImageSample()</code> .
La ventana no cierra con ESC	El foco no está sobre la ventana OpenCV o la GUI está congelada.	Hacer clic en la ventana y presionar ESC; si falla, terminar proceso desde terminal.

11. Extensión y Mantenimiento

11.1 Agregar nuevas resoluciones

Para agregar una resolución generada en cliente, modificar el diccionario `SIZES` y agregar una tecla en `MODE_KEYS`. La nueva resolución debe derivarse del frame nativo usando `cv2.resize()`.

Listing 21: Ejemplo de extensión de resolución.

```
SIZES["480p"] = (854, 480)
MODES.append("480p")
MODE_KEYS[ord("4")] = "480p"
```

Después de modificar, validar con:

```
python3 -m py_compile r1_camera_sdk.py
./start_camera.sh 480p enp0s31f6
```

11.2 Guardar frames en disco

Para depuración o generación de datasets, puede agregarse una tecla para guardar el frame actual.

Listing 22: Idea base para guardar frame actual.

```
if key == ord("s"):
    cv2.imwrite("r1_frame_debug.jpg", display)
    print("Frame guardado: r1_frame_debug.jpg", flush=True)
```

11.3 Agregar procesamiento de visión

El punto correcto para insertar algoritmos de percepción es después de `cv2.imdecode()` y antes de `cv2.imshow()`. En esa zona el frame ya está en formato BGR de OpenCV.

Listing 23: Ubicación recomendada para procesamiento adicional.

```
frame = cv2.imdecode(raw, cv2.IMREAD_COLOR)
if frame is None:
    continue

# Aquí insertar detección, tracking, filtros o inferencia.
processed = frame
cv2.imshow(window, processed)
```

11.4 Publicar frames a ROS2

Si se requiere integración con ROS2, una extensión viable es convertir el frame obtenido por RPC en un mensaje `sensor_msgs/Image` usando `cv_bridge`. Este enfoque evita depender de `/frontvideostream` y convierte la ruta operativa en una fuente ROS2 local controlada por el equipo.

11.5 Partes que no deben modificarse sin precaución

- `ChannelFactoryInitialize(0, iface)`: cambiarlo puede romper la comunicación DDS con el robot.
- `VideoClient.GetImageSample()`: es la llamada central al servicio de video.
- `CYCLONEDDS_URI`: debe apuntar a una configuración válida para la interfaz real.
- Conversión `np.frombuffer(bytes(data), dtype=np.uint8)`: debe mantenerse para decodificar correctamente el JPEG recibido.

12. Resumen de Actividades

Actividad	Estado	Observación
Identificación del flujo GStreamer previo	Completado	No se conserva como flujo principal para R1.
Implementación de visor con SDK2 Python	Completado	r1_camera_sdk.py es el script operativo.
Wrapper de lanzamiento	Completado	start_camera.sh simplifica modo, interfaz y CYCLONEDDS_URI.
Selección de resoluciones	Completado	720p, 360p, 180p y all.
Modo multi-vista	Completado	Genera mosaico local desde el frame nativo.
Ruta ROS2 /frontvideostream	Documentada como no recomendada	Se conserva el paquete ROS2 como referencia técnica, no como ejecución principal.
Validación física	Pendiente de registrar por prueba	Documentar interfaz final, IP y comportamiento real observado en el R1 de Robotics 4.0.
Extensiones futuras	Pendiente	Guardado de frames, publicación ROS2 local, detección de objetos o integración con navegación.

13. Anexo: Sustitución de Diapositivas de Visión

Para actualizar la presentación de herramientas del R1, la sección de visión debe reemplazar las referencias a GStreamer directo del G1 por el enfoque SDK2 Python.

13.1 Texto sugerido para diapositiva introductoria

Manejo de visión en R1

El flujo usado para la cámara frontal del R1 no se basa en un pipeline directo de GStreamer. La visualización se realiza mediante `unitree_sdk2_python`, usando `VideoClient.GetImageSample()` para solicitar frames JPEG al servicio DDS `videohub`. Los frames se decodifican con OpenCV y se muestran en tiempo real.

13.2 Texto sugerido para diapositiva de ejecución

```
cd ~/camara_r1_ws
chmod +x start_camera.sh
./start_camera.sh 720p enp0s31f6
./start_camera.sh all enp0s31f6
```

13.3 Texto sugerido para controles

Controles:
1 -> 720p
2 -> 360p
3 -> 180p
A -> todas las resoluciones
ESC -> salir

13.4 Código breve para imagen de diapositiva

```
client = VideoClient()
client.SetTimeout(3.0)
client.Init()

code, data = client.GetImageSample()
raw = np.frombuffer(bytes(data), dtype=np.uint8)
frame = cv2.imdecode(raw, cv2.IMREAD_COLOR)
cv2.imshow("R1 Camera", frame)
```

